ENGINEERING REPORT · RENDERING INTERCEPTION · GPU-NEURAL POST-PROCESSING

CRender

A Direct3D11 proxy-DLL framework that intercepts a game's rendering pipeline and re-synthesizes presented frames in retro styles — GPU ASCII/CRT shaders, CPU console wireframe, and a three-layer "Terminator Vision" pipeline backed by ONNX Runtime on DirectML with zero-copy D3D11↔D3D12 buffer interop.

Repository: F:\CRender · Revision: git 62a49cb (main) · Report date: 2026-10-19
Primary target: Unigine Heaven Benchmark 4.0 (x86 / D3D11) · Build: MSVC, C++20, /MT

Abstract

CRender is a drop-in `d3d11.dll` proxy for offline Direct3D11 applications. Loaded through the standard DLL search order, it forwards 49 of d3d11's 51 exports through architecture-specific trampolines, wraps the two device-creation entry points, and installs MinHook vtable detours across `ID3D11Device`, `ID3D11DeviceContext`, `IDXGIFactory(2)`, and `IDXGISwapChain(1,3)`. Inside the `Present` detour the framework applies a family of frame-restyling passes (pixel-art "RetroFX", a generated-glyph ASCII renderer, a Sobel/ONNX "Terminator" base, HUD and CRT lens layers) and — uniquely — it can force the game itself to render at 1/N resolution by rewriting swap-chain, texture, viewport and scissor state while preserving the game's believed screen size, so texel-space post pipelines remain correct. A dual-session YOLOv8 detector running on ONNX Runtime's DirectML EP feeds a lightweight IoU/centroid tracker that drives HUD target brackets; a pure-CPU mesh-AABB scanner acts as a zero-dependency fallback. The binary carries a static CRT and a KERNEL32-only import table; all optional components (ONNX Runtime, DirectML, D3D12) are resolved with `LoadLibrary` at runtime. This report documents the architecture, the interception and scaling machinery, the AI interop path, the tooling ecosystem, and the empirical findings collected while adapting the proxy to real engines.

Keywords — D3D11, DXGI, DLL proxying, vtable hooking, MinHook, ONNX Runtime, DirectML, D3D12 interop, post-processing, ASCII rendering, object detection, YOLOv8

Artifact	`d3d11.dll` (x86 + x64) — 51 exports, correct ordinals 1–51
Import surface	KERNEL32 + psapi only (static CRT `/MT`); ORT/DirectML/D3D12/d3dcompiler resolved at runtime
Modes	Off · NullPS · FlatPS · Wireframe · RetroFX · Ascii · ConsoleWire · Terminator
AI stack	ONNX Runtime 1.22.1 (DirectML EP) + DirectML 1.15.4; last NuGet release shipping win-x86
Scope	Offline/single-player titles only; graphics-layer interception (ReShade/DXVK category); no game-state writes, no anti-cheat interaction

Contents

1 **Introduction**

2 **Design goals and constraints**

3 **System architecture**

4 **The proxy layer**

- Export surface and ordinals
- Forwarding trampolines (x86 / x64)
- Lazy resolution of the real d3d11

5 **Interception layer**

- Vtable hooking strategy
- Hooked call surface
- Object lifetime safety
- Multi-device engines

6 **Render modes**

7 **Present-time post-processing**

- RetroFX shader
- Ascii shader and the glyph atlas
- Terminator base (HLSL/ONNX parity)
- Terminator lens

8 **RenderScale — low-resolution render injection**

9 **Neural pipeline — ONNX Runtime / DirectML interop**

10 **Detection, tracking and HUD**

- Dual-session YOLOv8 detector
- Weapon→owner sight lines
- Tracker
- Mesh-AABB fallback scanner
- HUD renderer

11 **Supporting infrastructure**

12 **Build system, deployment and tooling**

13 **Findings and lessons from real engines**

14 **Limitations**

15 **Safety, legal and ethical scope**

16 **Conclusion**

A **Appendix A — retro.json reference**

B **Appendix B — hooked vtable indices**

C **Appendix C — source map**

D **Appendix D — ONNX model graphs**

1 Introduction

CRender intercepts a Windows game's Direct3D11 pipeline at the graphics-API boundary and re-presents every frame through a stylization stage. It belongs to the same category of tools as ReShade or DXVK — injection at the graphics layer, below the game logic — but pursues a different goal: rather than subtle color grading or API translation, it synthesizes *visibly alternate representations* of the frame: a phosphor-terminal ASCII rendering, a pixel-art CRT look, a CPU-rasterized wireframe in a console window, and a "Terminator Vision" mode in which an ONNX network running on the GPU restyles the frame while a second network detects objects and a HUD draws target brackets over them.

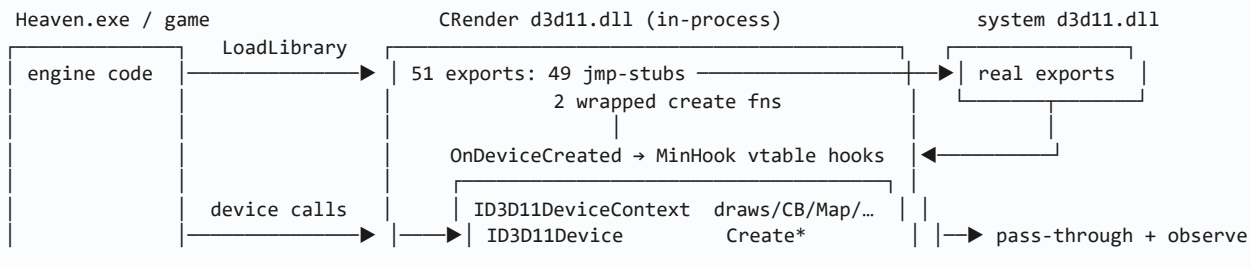
The project is engineered as a **drop-in proxy DLL**: a file named `d3d11.dll` placed next to the game executable is loaded by the normal Windows DLL search order — either as a static import or, as in the primary test target, through an explicit runtime `LoadLibrary`. No patching of the game binary, no driver installation, and no per-game recompilation are required; configuration lives in a single `retro.json` file beside the DLL.

This report documents the complete system as implemented at revision `62a49cb`: the forwarding machinery, the hook surface, every render path, the resolution-injection subsystem, the neural inference interop, the supporting tools, and the operational findings that emerged from running the proxy inside Unigine Heaven and probing it against several AAA engines' behaviors (multi-device engines, engine crash reporters, flip-model swap chains).

2 Design goals and constraints

Goal	Consequence in the implementation
Drop-in deployment	Named `d3d11.dll`; all 51 exports present with the correct ordinals so both import-by-name and import-by-ordinal loaders work.
Zero hard dependencies	Static CRT (`/MT`); import table is KERNEL32 + psapi only. ONNX Runtime, DirectML, D3D12 and d3dcompiler are `LoadLibrary`'d on demand — a bare proxy still runs every non-neural mode.
Survive hostile runtimes	Vectored exception handler for crash forensics, deferred COM release queue, owning (AddRef'd) captures, per-swap-chain device resolution, F10 panic unhook.
x86 and x64 from one source	x86 uses `__declspec(naked)` inline-asm trampolines; x64 (no `__asm` in MSVC) generates equivalent MASM stubs at CMake time from the same `FwdList.inc`.
Offline-only scope	Read-only observation of GPU traffic plus post-processing of the presented frame; no writes to game state, no input synthesis (input injection is documented as unreliable: `SendInput` is blocked in tested titles, `keybd_event` works).
Observable failure	Rotating log (`CRender.log`, 4 MB cap → `CRender.1.log`), 2 s stats line, linker `/MAP` so crash RVAs resolve without PDBs, per-component failure latches that degrade gracefully (ONNX → HLSL fallback).

3 System architecture



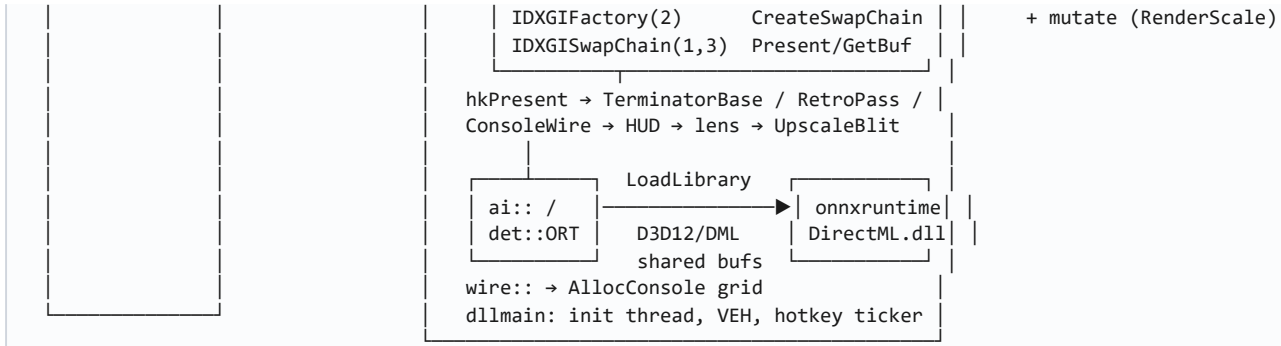


Figure 1 — Process view. Everything inside the box lives in one DLL loaded by the game; the ONNX/DirectML stack is optional and dynamically bound.

The codebase is layered. `src/proxy/` owns the export surface and resolves the real system DLL. `src/d3d11/` owns interception (`HookManager`), the post-process shaders (`RetroPass`), the console wireframe path (`ConsoleWire`), the geometry fallback scanner (`Targets`) and a shared pipeline-state snapshot type (`StateBlock`). `src/ai/` contains the two ORT-backed passes — `AiPass` (style base) and `Detect` (object detection) — plus `OrtApi`, the runtime binding shim. `src/term/` holds the HUD renderer and the track smoother. `src/core/` provides logging and the minimal JSON reader. A single entry point, `dllmain.cpp`, spawns a worker thread that performs all initialization outside loader lock and then acts as a 50ms hotkey/statistics ticker.

4 The proxy layer

4.1 Export surface and ordinals

The Windows SysWOW64/System32 `d3d11.dll` exports 51 functions. CRender reproduces the surface exactly: 49 pass-through exports enumerated in `src/proxy/FwdList.inc` (the source of truth is `tools/d3d11_exports_syswow64.txt`) and two wrapped creation functions, `D3D11CreateDevice` and `D3D11CreateDeviceAndSwapChain`. `exports.def` pins each export to its real ordinal (1–51); games that link by ordinal — or that call undocumented `D3DKMT*` entries — continue to work.

Two alternative designs were rejected: `.def`-file forwarders (`X=d3d11.X`) resolve to the proxy itself because the module is named `d3d11.dll`, producing infinite recursion; and shipping a renamed copy of the system DLL redistributes a version-locked Microsoft binary. Instead, each forwarded export is a tail-jump through a pointer slot (`g_fwd_<Name>`) resolved at runtime from the real DLL located by an absolute system path.

4.2 Forwarding trampolines (x86 / x64)

On x86 each export is a signature-free naked function; preserving the stdcall stack is automatic because the stub never touches arguments:

```
#define FWD_STUB(Name)                                      \
  extern "C" __declspec(naked) void Name()                  \
  {                                                        \
    __asm cmp dword ptr [g_fwd_##Name], 0                   \
    __asm jne short go_##Name                               \
    __asm call EnsureRealD3D11Impl   /* lazy resolve */     \
    __asm go_##Name:                                        \
    __asm jmp dword ptr [g_fwd_##Name] /* tail jump */      \
  }
```

MSVC has no `__asm` on x64, so `CMakeLists.txt` parses the same `FwdList.inc` and emits `fwd_stubs_x64.asm` — 49 MASM stubs. Because the lazy resolver may run inside an arbitrary call, the x64 stub preserves the integer

argument registers (rcx/rdx/r8/r9) and vector argument registers (xmm0–3, via `movdqu` into a 0x68-byte home region) around the resolver call before the unconditional `jmp [g_fwd_Name]`.

4.3 Lazy resolution of the real d3d11

`EnsurerealD3D11Impl` implements a four-state interlocked state machine (0 not-started → 1 loading → 2 ready / 3 failed; contenders spin on `SwitchToThread`). It is called eagerly from the proxy init thread so the game's first `D3D11CreateDevice` never pays the load cost, and defensively from any stub that is hit first. Resolution uses `GetSystemDirectoryW() + \d3d11.dll` — under WOW64 this correctly returns `SysWOW64`, so a 32-bit proxy always loads the 32-bit system DLL. The two wrapped exports additionally log arguments and capture the returned device/context/swap-chain for the hook layer.

5 Interception layer

5.1 Vtable hooking strategy

All interception is implemented with MinHook inline detours on *virtual-table slots*: the hook target is `*object -> vtbl -> entry[index]`. Because every instance of a COM implementation class shares one vtable, a single detour covers all contexts, devices or swap chains of that implementation — there is no per-object patching and no vtable rewriting. Indices are the published SDK layouts (a verified list is given in Appendix B); the one discovered empirically is `DrawAuto` at context slot 38.

Swap chains created through `D3D11CreateDeviceAndSwapChain` are captured in the wrapped export. Engines that instead call `IDXGIFactory::CreateSwapChain` directly — Heaven does — are caught by walking `device → IDXGIDevice → IDXGIAdapter → IDXGIFactory(2)` inside `OnDeviceCreated` and hooking `CreateSwapChain` / `CreateSwapChainForHwnd`.

5.2 Hooked call surface

The installed hooks fall into four functional groups: **counters/provenance** (`Map`, `Unmap`, `IASet*`, `PSSetShaderResources`), **state tracking** (`VSSetConstantBuffers`, `UpdateSubresource`, `RSSetState/Viewports/ScissorRects`, `OMSetRenderTargets(+UAVs)`, `PSSetShader`), **draw interception** (`Draw`, `DrawIndexed`, `Draw(Indexed)Instanced`, `DrawAuto` — where skip decisions, mesh readback and target-scan fan-out happen), and **creation/present** (device `Create*` methods for buffer AABB capture, PS bytecode hashing and texture shrinking; factory swap-chain creation; swap-chain `Present` / `Present1`, `GetBuffer`, `ResizeBuffers(1)`). Roughly thirty slots are detoured in total.

5.3 Object lifetime safety

Releasing a COM interface inside `OnDeviceCreated` runs its destructor on the game's `D3D11CreateDevice` call stack — Yakuza0 faulted inside dxgi when a second (BGRA/D2D) device displaced the first. The fix is a 16-slot *deferred release queue*: superseded captures are parked and drained from `hkPresent` / `Shutdown`, on a safe call stack. A full queue deliberately leaks rather than releasing in place. Captures are *owning* (AddRef'd) and a probe device (created without an immediate context for capability queries and released immediately) never displaces a live context-bearing capture.

5.4 Multi-device engines

Some engines (observed: WatchDogs2) create multiple devices — probe/loading devices alongside the real one. `PresentCommon` therefore never trusts the captured globals for rendering work: it calls `sc->GetDevice()` / `GetImmediateContext()` on the presenting chain and runs all retro passes on that pair, releasing both references afterward.

6 Render modes

Mode	Mechanism	Where it runs
`Off`	All detours pass through; only counters/stats remain.	—
`NullPS`	`PSSetShader(nullptr)` — geometry and depth still render; diagnostic mode.	per-draw, in `hkPSSetShader`
`FlatPS`	A compiled constant-color pixel shader substitutes every game PS.	per-draw
`Wireframe`	`RSSetState` swaps in a wireframe rasterizer state.	per-draw
`RetroFX`	Fullscreen post: pixelate + 4×4 Bayer ordered dither + 3-bit palette + scanlines + CRT barrel + vignette.	`hkPresent`
`Ascii`	Cell-grid post: 4-tap luminance → generated 10-glyph density atlas; five phosphor palettes.	`hkPresent`
`ConsoleWire`	Budgeted VB/IB readback → CPU projection (world·view·proj) → Bresenham edges with slope-matched `−|\╱` chars and depth fade into an `AllocConsole` window at `ConsoleHz`.	draw-time collect + present-time raster
`Terminator`	Three layers: ONNX (ORT DirectML) or HLSL Sobel base → instanced-glyph HUD (text, reticle, target brackets, sight lines) → CRT lens. Config alias `"Neural"` maps here.	`hkPresent`

F9 cycles the user-facing subset `Off → RetroFX → Ascii → Terminator`; the debug modes remain reachable via the config `"Mode"` key. `retro::StateBlock` snapshots and restores every pipeline slot the present-time passes touch (shaders + class instances, SRVs, samplers, CBs, RT/DS, blend/depth/rasterizer, viewports), so a frame ends bit-identical to how the game left it.

7 Present-time post-processing

All fullscreen effects share one `SV_VertexID` triangle vertex shader and a 48-byte parameter cbuffer carrying the drawn-region size, letterbox offset, and effect tunables. Effects run at *output* resolution sampling the (possibly shrunken) scene texture — so ASCII glyphs and scanlines stay crisp when `RenderScale` is active.

7.1 RetroFX

`kPsRetroSrc`: NDC-space barrel distortion `uv' = uv·(1 + crt·r²)`, block quantization to `pixelPx` cells, a 4×4 Bayer ordered-dither matrix feeding a `floor(c·7+t)/7` 3-bit-per-channel palette, a per-block sine scanline (fine 2-px fallback when `pixelPx < 3`), and a smoothstep vignette.

7.2 Ascii shader and glyph atlas

`kPsAsciiSrc` divides the output into `CellPx`-sized cells, takes a 4-tap luminance sample per cell, lifts darks (`pow(lum,0.7)·1.1`) so the whole ramp is used, and selects one of ten density glyphs — the bitmap ramp `' ' '.' ':' '-' '=' '+' '*' '#' '%' '@'` generated procedurally into an R8 atlas at startup (exact texel fetch via `atlas.Load`, no sampler bleed). Palettes: **green**, **amber**, **mono**, **cyan**, and **color** (divides the scene color by its own luminance to recover hue+saturation). A faint `bg = phosphor·lum·0.07` fill preserves silhouettes between glyph strokes. The same atlas' second half holds 5×7 text glyphs shared by the HUD.

7.3 Terminator base — ONNX/HLSL parity

The HLSL fallback (`kPsTermBaseSrc`) replicates the bundled `terminator_vision.onnx` model exactly: luma → 3×3 Sobel with an explicit zeroed 1-px border (ONNX `Pad` modes weren't in the op budget, so the model multiplies by a constant mask instead) → magnitude `√(gx²+gy²+ε)` → soft threshold `sat(e·2−0.05)` scaled by `TermEdgeGain` → output encoding `R = 0.525·Y·tint + e`, `G = B = 0.7·e` — the crimson-void/white-outline

look. Because the ONNX and shader paths compute identical math, `TermEdgeGain` / `TermTint` re-grade whichever base is active.

7.4 Terminator lens

`kPsLensSrc`: the same CRT barrel, a 3-px-period cosine scanline (`0.5+0.5·cos(y·2π/3)`), and a red-biased vignette (`RGB × (1−0.55v, 1−v, 1−v)`) that sinks corners to deep red rather than black.

8 RenderScale — low-resolution render injection

Restyling alone leaves the game shading every native pixel. `RenderScale N` makes the game *itself* render at 1/N while believing the screen is still native — a genuine performance lever, not just an aesthetic. The mechanism is a consistent fiction maintained across five subsystems:

- **Swap chain** — `hkCreateSwapChain` / `hkCreateSCForHwnd` rewrite `BufferDesc` to believed/N and set `STRETCHED` scaling. The real backbuffer is small; DXGI stretches it at present. Shrinking the *backbuffer itself* (rather than only intermediate RTs) is the load-bearing decision: engines like Unigine post-process with texel-space `Load()`, which stays consistent only if the backbuffer actually is low-res. The earlier RT-only variant rendered black.
- **Textures** — `hkCreateTexture2D` shrinks RT/DS-bindable textures that are screen-sized *or a proportional divisor of it* (½/¼-res bloom passes scale too), while shared/GDI/cube/uploaded textures are contract-sized and left alone; mip counts are clamped to the shrunken maximum; failed shrinks retry at native size and log. A 1024-slot open-addressed table (`FindScaled/SetScaled`) records each resource's believed dims.
- **Viewports & scissors** — `hkRSSetViewports` / `ScissorRects` and `NoteBoundTargets` re-push the game's state divided by N whenever a scaled target is bound.
- **Copies** — `hkCopySubresourceRegion` remaps dst offsets and src boxes into whichever side's space is scaled.
- **Present** — for blit-model chains a second, windowed native-size chain (`EnsurePresentChain`) point-samples the small buffer upward as true integer scaling and presents *it* instead — bypassing DXGI's bilinear stretch; failure latches off and falls back to the game's own Present without surfacing an error. For *flip-model* chains the backbuffer can't be shrunk, so `hkGetBuffer` substitutes a small proxy texture the game renders into, which is then upscaled. In exclusive fullscreen the extra chain is never presented (DXGI would fight over the output).

Companion instrumentation: `SkipShadows` drops depth-only passes onto non-screen DSVs; `SkipPS` blocklists pixel shaders by a bytecode hash recorded at `CreatePixelShader` (`PassDump` discovers hashes by logging `ps=%016llx rt=WxH` per pass); `ShaderStrip.SkipMaxIndexCount` / `SkipOrthographic` drop fullscreen/Ortho-UI quads; `GpuBurn N` adds N synthetic heavy fullscreen passes to emulate a weaker GPU for benchmarking; `PatchScreenCB` rewrites screen-dim floats in CB uploads for engines whose SSAO/deferred passes misbehave. Guards (`g_inRetroPass`, `g_inSCCreate`) keep the proxy's own draws and DXGI-internal creations out of the scaling/skip logic. RenderScale must be configured *before* device creation (launch-time only — the scale is latched on first device).

9 Neural pipeline — ONNX Runtime / DirectML interop

`src/ai/AiPass.cpp` runs the Terminator base style network entirely on the GPU with **zero CPU copies**. `onnxruntime.dll` / `DirectML.dll` are `LoadLibrary`'d from the proxy directory (ORT1.22.1 DirectML EP — the last NuGet shipping win-x86 — plus DirectML1.15.4), resolved through `OrtApi.cpp`; nothing links statically, preserving the KERNEL32-only import table.

- **Interop buffers** — three linear fp32 buffers (input, plus a ping-pong output pair) are created as *shared D3D12 committed resources* on the game's adapter and opened on the D3D11 device. D3D11 writes the

NCHW-planar input through a UAV pixel pass and reads the output via SRV; ORT binds the identical memory with `CreateGPUAllocationFromD3DResource`.

- **Synchronization** — a D3D11 event query drains the D3D11 queue before `Run()`; a D3D12 fence wait after Run orders DML's work against the next D3D11 pass.
- **Model contract** — fixed-shape `[1,3,AiH,AiW]` fp32 NCHW (dynamic shapes rejected); `AiWidth % 64 == 0` for D3D12 texture-copy row alignment. Any inference failure latches the ONNX path off until F11; the HLSL base takes over seamlessly.
- **Temporal modes** — `AiTemporal` 0: single-input; 1: `lerp(current, prevOut, AiBlend)` using the ping-pong buffers; 2: two-input models (`retro_ai_temporal.onnx`) with the previous output bound as input1.
- **Placement in frame** — runs inside `TerminatorBase` on the (possibly shrunken) game backbuffer, after the detector has sampled the *raw* frame and before HUD/lens/upscale.

Model tooling produces all shipped networks with DirectML-friendly ops only (Conv, Mul, Add, Sqrt, Clip, Concat — no control flow, no dynamic axes): `retro_ai.onnx` (7.7KB pixel-art: blockify→saturate→contrast→posterize), `terminator_vision.onnx` (the Sobel encoding of §7.3), `yolov8n.onnx` and the weapon-tuned `best.onnx` / `det_weapons.onnx` (≈12MB YOLOv8 class detectors). Any fp32 NCHW `[1,3,H,W]` style-transfer ONNX can be dropped in as `retro_ai.onnx`.

10 Detection, tracking and HUD

10.1 Dual-session YOLOv8 detector

`src/ai/Detect.cpp` runs an *independent* D3D12/DML/ORT stack (it works even when the style base is the HLSL fallback) on the raw backbuffer, sampled before any restyle. Up to two detector sessions are supported — session0 typically `yolov8n.onnx` (COCO classes by name via `DetClasses` / `DetWeapon`), session1 a numeric-id model such as `det_weapons.onnx` (0=guns, 1=knife) whose ids are offset +1000 so they never collide. An optional third stage is a micro-classifier scoring `P(weapon|crop)` on person crops with its own keep-threshold (`DetClsConf`). The decoder handles YOLOv8-style `[1, 4+K, N]` outputs (including the OBB variant's extra angle row), maps boxes back through the letterbox rect, applies per-class NMS, honors `DetStride` (infer every Nth present), and can dump input PPMs (`DetDump`) for offline inspection.

10.2 Weapon→owner sight lines

`DetSight` draws dotted normalized screen-space rays from each weapon box toward its owner — the `person` track that contains it, else the nearest person — extended by `DetSightLen` beyond the weapon. Detection labels render as COCO names, `GUN` / `KNIFE`, or `DET nn`.

10.3 Tracker

`src/term/Tracker.cpp` is a 32-track smoother operating on normalized boxes: score-ordered greedy association (IoU ≥ 0.25 primary, centroid distance < 0.08 fallback), EMA smoothing (α=0.35), coasting up to 0.35s without matches, a 1.2s lock-in threshold, and monotonic 1–99 ids for the `TGT nn` display. A hit counter confirms genuine tracks. One instance per producer thread — no locking needed.

10.4 Mesh-AABB fallback scanner

When no detector model exists, `src/d3d11/Targets.cpp` produces brackets with zero GPU readback: at `CreateBuffer` time it computes local AABBs of each vertex buffer's initial data for all 14 plausible strides (12–64B, finite-checked, degenerate-extent rejection) in an 8192-slot open-addressed table; `UpdateSubresource` writes give it the 64B per-draw world CB and the 144B camera CB (proj@0, view@64, perspective validated by $|m_{11}|>0.5$). Per scene draw, the 8 box corners are projected (row-vector `v·world·view·proj`), near-plane/off-screen/ortho/UI and size outliers (2–30% screen area) are rejected, same-mesh duplicates collapse

at IoU>0.8, and the best area-scored boxes feed the same Tracker. Dynamic/late-filled buffers simply never produce brackets.

10.5 HUD renderer

`src/term/Hud.cpp` draws the overlay as **one instanced draw call** of quads from the shared retro glyph atlas — instances are tagged glyph/rect/ring with per-instance color, alpha blended onto the presenting target: scrolling diagnostic text columns (real status lines such as `AI ONNX DML`, `DET ONNX`, `TGT n`), a center reticle with drifting coordinates, and animated target brackets. Every element is gated by config (`TermHud*`, `HudMaxTargets`, `TermHudSpeed`).

11 Supporting infrastructure

- **Entry & ticker** — `DllMain` does nothing but `DisableThreadLibraryCalls` + `CreateThread` (loader-lock discipline). The thread resolves the real d3d11 eagerly, loads config once, then loops at 50ms on `HotkeyTick`/`StatsTick`, re-snapshotting module ranges every ~3.2s.
- **VEH crash forensics** — engines with their own SEH/UEF reporters swallow faults silently. A first-in-chain vectored handler logs hard faults only (severity 0xC…; stack overflow skipped — no stack left to log on), deduplicates faulting IPs, resolves module+RVA against a snapshot taken on the ticker thread (never inside the handler — loader-lock deadlock), and scans 128 stack slots for probable return addresses. The linker emits a `.map` so RVAs symbolize offline.
- **Hotkeys** — edge-triggered `GetAsyncKeyState` with a foreground-window PID check: **F8** palette, **F9** mode cycle, **F10** panic (`MH_DisableHook(ALL)` toggle), **F11** config reload, **F12** RenderDoc capture via `RENDERDOC_GetAPI(10100)` when renderdoc.dll is resident.
- **Config** — `retro.json` next to the DLL, parsed by a zero-dependency strstr-style scanner (`src/core/Config.h`); `ExtractObject` lifts nested preset blocks (`TerminatorLens`) so key names can't collide; `"Neural"` is accepted as an alias for Terminator. Hot-reloadable via F11 except launch-latched keys (`RenderScale`).
- **Logging** — `CRender.log` beside the DLL, timestamp + TID + severity + tag, `_SH_DENYNO` (tail-able live), 4MB rotation to `CRender.1.log`, `OutputDebugString` fallback. Stats every 2s: `mode fps cpuFrameMs presents draws maps skipped scale aiMs aiRuns`.
- **Shutdown** — orderly: ticker stopped, VEH removed, `MH_DisableHook(ALL)` → vtable-patch restore → `MH_Uninitialize`, then all retro/AI/detector/HUD/target/staging/chain objects released; skipped entirely on process termination (`reserved != NULL`).

12 Build system, deployment and tooling

12.1 Build

```
cmake --preset win32-msvc          # VS 18 2026 generator, arch=Win32
cmake --build --preset win32-release
#  → build/win32/Release/d3d11.dll + test_cube.exe
# x64 mirror: presets x64-msvc / x64-release → build/x64/  (generated MASM stubs)
```

C++20, `/W4 /WX /permissive- /utf-8`, static CRT. The target is named `retro_proxy` so its import lib can't shadow the SDK's `d3d11.lib`; the produced binary is still `d3d11.dll`. MinHook builds in-tree as a subdirectory. The ORT headers/DLLs are probed at configure time and gate `RETRO_HAVE_ORT` — without `third_party/onnxruntime` the DLL still compiles and Terminator logs a fallback warning. A POST_BUILD step deploys the DLL, `retro.json`, ORT/DirectML runtimes and ONNX models to `HEAVEN_BIN` when present. Two Dockerfiles containerize what's containerizable: `tools.Dockerfile` (Linux CPU-ORT for model tooling) and

`windows-build.Dockerfile` (Windows Server Core + VS Build Tools, x86+x64 artifacts via `docker cp`); the runtime itself requires a real GPU desktop and game.

12.2 Tooling

Tool	Purpose
`tools/CRender_wizard.py`	PyQt6 GUI installer — pick a game folder, Apply deploys proxy+config+models, Rewind restores backups.
`tools/crenderctl.ps1`	CLI equivalent: `-Enable/-Disable/-Status`, `-Neural` adds ORT/DML + models; tracks installs via `.crenderctl.json`.
`tools/screen_vision.py`	Desktop port of the Terminator/ASCII pipeline — mss capture + PyQt6 transparent click-through HUD over the live screen (F8/F9/F10/F11/End).
`tools/test_cube/`	Minimal windowed D3D11 app — rotating cube, title-bar FPS, F1 toggles a heavy/light pixel shader.
`tools/ai/fetch_ort.ps1`	Walks NuGet versions newest-first to find the last ORT-DML release still shipping win-x86; pulls matching DirectML.
`tools/ai/make_*.py, fetch_detector.py`	Generate `retro_ai(.onnx/_temporal)`, `terminator_vision.onnx`, export Ultralytics `.pt` → fixed-shape `yolov8n.onnx`.
`tools/ai/{verify_terminator,sanity_check,test_bestgun,wrap_style}.py`	Model verification, ORT environment check, weapon-detector eval, style-model wrapping.
`tools/ai/render_model_graph.py`	Renders an ONNX op-graph as a layered DAG PNG (longest-path layering + barycenter ordering); used for Appendix D.

13 Findings and lessons from real engines

Adapting the proxy beyond the test cube surfaced a set of non-obvious engine behaviors; each is encoded in the code as a guard or fallback.

> **F1 — BACKBUFFER SHRINK, NOT RT SHRINK**
> RenderScale had to shrink the *swap-chain backbuffer* (believed dims kept, STRETCH scaling). Shrinking only intermediate RTs blacks the screen: Unigine's HDR→LDR composite reads its sources with texel-space `Load()` at believed coordinates, which only matches when the final target actually is low-res. Engines sampling via `Sample()` +UV would have survived the naive version.

F2 — FLIP-MODEL CHAINS CAN'T SHRINK

`FLIP_*` swap effects reject shrunken backbuffers. `hkGetBuffer` instead returns a small proxy texture the game renders into; that texture is upscaled at present. Blit-model chains get the stronger path — a second windowed native-size chain presents a point-sampled integer upscale, bypassing DXGI bilinear stretch entirely.

F3 — DEFERRED COM RELEASE

Releasing a superseded device/context inside `D3D11CreateDevice` runs the destructor on the game's stack — Yakuza0 AV'd in dxgi. All captures are AddRef'd and old ones drained from `hkPresent`. Probe devices (no immediate context) never displace a live capture.

F4 — NEVER TRUST THE CAPTURED DEVICE FOR RENDERING

Multi-device engines (WatchDogs2) leave the globals bound to a probe device. `PresentCommon` resolves device+context from the presenting swap chain itself.

F5 — ENGINE CRASH REPORTERS HIDE FAULTS

UEF-style reporters (Ubisoft's) swallow exceptions — no WER event, no dump, not even `DLL_PROCESS_DETACH`. A first-in-chain vectored handler with pre-snapshotted module ranges is the only reliable observer; stack-overflow faults are deliberately skipped (no stack to log on).

F6 — HEAVEN'S CONTRACT, DISCOVERED

The launcher PE is 32-bit and the engine (`Unigine_x86.dll`) loads `d3d11.dll` via `LoadLibrary`; device created with `D3D11_CREATE_DEVICE_SINGLETHREADED` and an explicit adapter; the swap chain arrives via `IDXGIFactory::CreateSwapChain`, not the AndSwapChain export; constant buffers are written with `UpdateSubresource`, never `Map`; the camera CB is 144B at VS slot0 (proj@0: m_{00}=0.9743, m_{11}=1.7321, m_{22}=−1, m_{23}=−1, m_{32}=−0.3 → z_n=0.3, far≈∞, RH row-major) while a 64B ortho CB shares the slot for UI — ortho detection must read matrix contents, not the slot. `DrawAuto` = vtbl[38]. `SendInput` is blocked; `keybd_event` works.

F7 — ORT AVAILABILITY DICTATES ARCHITECTURE

ONNX Runtime's DirectML NuGet dropped win-x86 after 1.22.1 — precisely the architecture the primary target needs. `fetch_ort.ps1` encodes the version walk; the runtime is LoadLibrary'd so the proxy degrades to HLSL when DLLs are absent rather than failing to load.

F8 — PRESENT IS THE ONLY SAFE WORK QUEUE

`DXGI_PRESENT_TEST` occlusion probes and nested Present/Present1 routes are excluded via a thread-local guard; a failed *self-owned* upscale chain is never reported to the game (engines treat a failed Present as device loss); second windowed chain is suppressed under exclusive fullscreen.

F9 — EXPORT PARITY IS LOAD-BEARING

All 51 exports with correct ordinals matter: games import D3DKMT* entries by ordinal, and the wrapped-vs-forwarded split (49+2) keeps every call signature-free except the two creation points where capture must happen.

14 Limitations

- **Launch-time scaling** — `RenderScale` is latched at first device; F11 cannot re-scale live (RTs/backbuffer are created once).
- **Heuristic pass skipping** — shadow/ortho/index rules can misfire on unusual engines; `PassDump` + `SkipPS` are the manual escape hatch.
- **AABB scan coverage** — only buffers with CPU initial data get brackets; dynamic/streamed geometry is invisible to the fallback.
- **Fixed-shape models only** — dynamic axes rejected; `AiWidth` must be a multiple of 64.
- **x86 ORT ceiling** — neural features on 32-bit games are frozen at ORT1.22.1/DML1.15.4.
- **Single-context assumption in trackers** — geometry/target collection keys off the immediate context; deferred-context games get post modes but weaker brackets.
- **CPU fallback cost** — ConsoleWire's readback path is budgeted (8-slot staging pool, per-frame draw caps) but still taxes bandwidth-heavy titles.

15 Safety, legal and ethical scope

The project targets **games you own, offline, with no anti-cheat**. It observes GPU traffic and rewrites the presented image; it does not read or write game state, synthesize input, or interact with protection systems — the same operational envelope as ReShade or DXVK. Detection features are presentational (brackets drawn on the output frame) and provide no feedback channel into the game.

16 Conclusion

CRender demonstrates that a self-contained, dependency-free proxy DLL can host a full frame-synthesis stack — shader effects, neural style transfer, and real-time detection-driven overlays — inside a shipping-class D3D11 engine without modification. Its most transferable engineering results are the believed-vs-real backbuffer scaling technique, the lifetime/present-queue disciplines that keep a proxy stable inside hostile engine runtimes, and the shared-buffer D3D11 ↔ D3D12 interop that makes GPU inference practical from inside an injected 11-on-12-free process. Natural extensions are richer per-game profiles under `config/games/`, additional bundled detectors, and D3D11On12-free zero-copy paths for newer ORT releases on x64.

A Appendix A — retro.json reference

Key	Default (shipped)	Meaning
Mode	"Ascii"	Off · NullPS · FlatPS · Wireframe · RetroFX · Ascii · ConsoleWire · Terminator ("Neural" = Terminator)
Hotkeys.*	F8/F9/F10/F11	Palette / mode cycle / panic unhook / config reload
Output.*	120×40 @30Hz	ConsoleWire grid size + blit rate
PixelPx / CellPx	4 / 8	RetroFX block size / Ascii cell size (clamped ≥ RenderScale)
Crt / Vignette	0.12 / 0.35	RetroFX barrel + vignette
Palette	"color"	green · amber · mono · cyan · color
DumpMesh / WireDraws / WireCull	0 / 4 / 1	Mesh readback budget, draws/frame, backface cull
RenderScale	4	Scene resolution divisor (launch-time only)
PatchScreenCB	0	Rewrite screen-dim floats in CB writes
SkipShadows / SkipPS / ShaderStrip	1 / [] / 6+ortho	Pass skipping: shadow maps, PS hash blocklist, tiny/ortho draws
PassDump / GpuBurn	0 / 0	Per-pass PS discovery log / synthetic GPU load passes
DumpCBs	0	Recon: dump N small CB writes (matrix-slot hunting)
AiModel / AiWidth / AiHeight	terminator_vision.onnx / 640 / 360	Style model + fixed input shape (width %64)
AiTemporal / AiBlend / AiStride / AiDevice	0 / 0.5 / 1 / 0	Temporal mode, blend, frame stride, adapter pick
TermBase / TermBaseOnnx	false/false	Enable restyle base; prefer ONNX over HLSL
TermHud / TermHudText / TermBrackets / TermReticle / TermHudSpeed	t/f/t/f/1.0	HUD elements + animation rate
TermLens / TerminatorLens{Crt,Vignette,Scanlines}	false / 0.1,0.45,0.35	Lens pass + preset
HudMaxTargets / TermEdgeGain / TermTint	6 / 1.0 / [1,0,0]	Bracket cap, edge gain, void tint
DetModel / DetModel2 / (+model3)	yolov8n.onnx / best.onnx	Detector sessions 0–2 (classifier)
DetConf / DetClsConf / DetStride / DetDump	0.15 / 0.80 / 2 / 0	Confidence, stage-2 threshold, infer stride, PPM dumps
DetClasses / DetWeapon / DetClasses2 / DetWeapon2	person / "" / 0 / 0	Class filters (COCO names or ids) + weapon flags
DetSight / DetSightLen	false / 0.55	Weapon→owner rays + length
Geometry.{MaxCacheMB,MaxBufferBytes,StagingDelayFrames}	256 / 32MB / 3	Mesh readback cache/staging budgets

B Appendix B — Hooked vtable indices

Interface	Slot → Method
ID3D11DeviceContext	7 VSSetConstantBuffers · 8 PSSetShaderResources · 9 PSSetShader · 12 DrawIndexed · 13 Draw · 14 Map · 15 Unmap · 18 IASetVertexBuffers · 19 IASetIndexBuffer · 20 DrawIndexedInstanced · 21 DrawInstanced · 33 OMSetRenderTargets · 34 OMSetRenderTargetsAndUnorderedAccessViews · 38 DrawAuto · 43 RSSetState · 44 RSSetViewports · 45 RSSetScissorRects · 46 CopySubresourceRegion · 48 UpdateSubresource
ID3D11Device	3 CreateBuffer · 5 CreateTexture2D · 9 CreateRenderTargetView · 10 CreateDepthStencilView · 11 CreateInputLayout · 15 CreatePixelShader
IDXGIFactory / 2	10 CreateSwapChain · 15 CreateSwapChainForHwnd
IDXGISwapChain / 1 / 3	8 Present · 9 GetBuffer · 13 ResizeBuffers · 22 Present1 · 39 ResizeBuffers1

C Appendix C — Source map

File	Contents
src/dllmain.cpp	DllMain + init thread, VEH crash forensics, module snapshotter, 50 ms ticker
src/proxy/{ProxyDll,ProxyExports}.cpp, FwdList.inc, exports.def	Real-DLL resolution state machine, 49 forwarders + 2 wrapped exports, ordinals
src/d3d11/HookManager.cpp (~3k lines)	All MinHook detours, RenderScale machinery, mode dispatch, config load, hotkeys, stats, present orchestration
src/d3d11/RetroPass.cpp	Fullscreen VS + RetroFX/Ascii/TermBase/TermLens pixel shaders, glyph atlas, upscale blit, GpuBurn, StateBlock usage
src/d3d11/ConsoleWire.cpp	Console wireframe: geometry collect, projection, Bresenham raster, console blit
src/d3d11/Targets.cpp	VB AABB capture, world/view/proj tracking, corner projection, candidate feed
src/ai/AiPass.cpp	ORT-DML style pass: D3D12 shared buffers, session, interop binding, sync, temporal
src/ai/Detect.cpp	Independent ORT stack: ≤3 sessions, YOLOv8/OBB decode, NMS, sight lines, classifier
src/ai/OrtApi.cpp	LoadLibrary binding of onnxruntime/DirectML/D3D12 entry points
src/term/{Hud,Tracker}.cpp	Instanced-quad HUD; IoU/EMA tracker
src/core/{Config.h,Log.cpp}	strstr JSON reader + ExtractObject; rotating logger
tools/	Wizard GUI, crenderctl.ps1, screen_vision.py, test_cube, ai/* model tooling
config/	retro.json, ONNX models, games/Heaven.exe.json recon profile
third_party/	MinHook (submodule), onnxruntime (fetched), style models

D Appendix D — ONNX model graphs

Op-level DAGs of the six shipped models, rendered by `tools/ai/render_model_graph.py` (longest-path layering, barycenter edge-ordering; colors: blue = convolutions, amber = activations/clips, indigo = tensor reshaping ops, green = reductions, green/red terminals = graph inputs/outputs).

Model	Nodes	Edges	Layers	Dominant ops	Role
terminator_vision.onnx	18	19	15	Mul×5 · Conv×3 · Add×3 · Clip×3	Terminator base style (Sobel/luma encoding)
retro_ai.onnx	70	87	30	Mul×18 · Conv×11 · Sub/Add×8	Pixel-art placeholder style (blockify→posterize)
retro_ai_temporal.onnx	74	91	32	Mul×20 · Conv×11 · Add×9	Two-input temporal variant of retro_ai
yolov8n.onnx	235	322	190	Conv×64 · Sigmoid×58 · Mul×58 · Concat×17	Detector session 0 (COCO classes)
det_weapons.onnx	263	360	197	Conv×64 · Mul×60 · Sigmoid×58 · Constant×22	Detector session 1 (weapon ids 0/1)
best.onnx	263	360	197	same family as det_weapons	Tuned weapon detector (session 1 alternative)

The full-size graphs are shipped as standalone vector files in `docs/images/`:

File	Model graph
docs/images/1-terminator_vision.svg	Terminator base style — frame → luma Conv → border-masked Sobel Conv → magnitude → soft-threshold edges → per-channel Clip → `Concat(R=bg+e, G=B=0.7e)` → styled. Matches the HLSL fallback in §7.3 op-for-op.
docs/images/2-retro_ai.svg	Single-input pixel-art style chain (blockify → saturate → contrast → posterize).
docs/images/3-retro_ai_temporal.svg	retro_ai body plus the second `prev` input feeding the temporal blend.
docs/images/4-yolov8n.svg	YOLOv8 backbone → neck concat/split fan-out → three-scale detection head.
docs/images/5-det_weapons.svg	Same YOLOv8 family; head emits gun/knife ids consumed as session-1 classes +1000.
docs/images/6-best.svg	Tuned weapon checkpoint, structurally identical to det_weapons.

Report generated from source at revision 62a49cb. All behavioral claims are grounded in the cited files; the shipped `retro.json` values are shown as defaults. Model graphs ship separately as `docs/images/{1..6}-*.svg` (also as `modelgraph_*.png`).